

ASSESSMENT AND INTERNAL VERIFICATION FRONT SHEET (Individual Criteria)

(Note : This version is to be used for an assignment brief issued to students via Classter)

Course Title	MCAST Bachelor of Science (Honours) in Digital Games Development (IT6-03-19)		Lecturer Name & Surname	Kris Domancich	
Unit Number & Title		ITMSD-606-1610-Sound Engineering			
Assignment Number, Title / Type		Adaptive music project			
Date Set		28 th April 2026	Deadline Date	22 nd May 2026[	
Student Name			ID Number		Class / Group

Assessment Criteria	Maximum Mark
AA1.3 – Apply Equalisation filters using pillars and/or pre-sets	7
KU2.1 - Identify and report different sounds in a track.	5
KU3.3 - Distinguish/clarify the need of Audio or/and MIDI effects.	5
KU4.2 - Identify the role of interactive composition in dynamic music.	5
AA4.3 - Develop automation in an audio project.	7
SE4.4- Evaluate the use of proper sound dynamics within a context.	10
SE4.5 - Justify different techniques learnt and present alternatives or improvements	10
KU1.1 – Explain track composition in digital application workstation	5
KU1.2 - Identify and order different sounds within the frequency spectrum.	5
KU1.4 - Examine parametric Equalisation within a context.	5
KU4.1 - Present a sound brief to explain an audio project within a gaming strategy.	5
AA2.2 - Apply routing processes between channels.	7
AA3.1 - Produce realistic sounds with the use of VSTs/Plug-ins.	7
AA3.4 - Investigate the use compressors/expanders within an audio track.	7
SE3.2 - Manage sound improvements and implementations within a context.	10
Total Mark	100

Notes to Students:

- This assignment brief has been approved and released by the Internal Verifier through Classter.
- Assessment marks and feedback will be available online via Classter (<http://mcast.classter.com>) following release by the Internal Verifier
- Students submitting their assignment on Moodle/Unicheck will be requested to confirm online the following statements:

Student's declaration prior to handing-in of assignment

- ✚ I certify that the work submitted for this assignment is my own and that I have read and understood the respective Plagiarism Policy

Student's declaration on assessment special arrangements

- ✚ I certify that adequate support was given to me during the assignment through the Institute and/or the Inclusive Education Unit. ❖ I declare that I refused the special support offered by the Institute.

Sound Engineering

Bachelors of Science (Hons.)

Adaptive music project

Assignment Guidelines

Read the following instructions carefully before you start the assignment. If you do not understand any of them, ask your lecturer.

- This assignment must be attempted by each **student and** all tasks should be completed **individually**. Note that it consists individual sections.
- Note that you must **submit your assignment via Moodle** using the assignment upload facility. You will need to upload a compressed (.zip) file with your name, surname and class.
- **Copying and plagiarism** are strictly **prohibited** and will be penalised in line with the **College's disciplinary procedures**. Any students caught copying during the exam will be disqualified immediately and corrective action will be taken in line with the college disciplinary procedures.
- Make sure you make use of both VLE and OneDrive for packs, clips and samples to help you. You may also make use of third-party websites.
- Upload all material on VLE assignment upload facility when done.
- This assignment carries a total of **100%** of the overall unit.

Scenario

Your task for this assignment is to **design, document** and **implement** the complete music and sound package for a game of your choice. The game must be built in either Unity or Unreal Engine and must target either macOS or Windows as its delivery platform. Your soundtrack must be adaptive and must

clearly demonstrate how music and game-audio interact with distinct phases of gameplay (for example: exploration, tension / build-up, combat, resolution, menus, or any other states appropriate to the genre you choose).

You are expected to use at least two contrasting musical genres or moods across the adaptive score — for example a cinematic / orchestral state paired with a higher-energy electronic state — and to justify your choices against the game's narrative or mechanics.

You are required to use the following software, and you are responsible for ensuring they operate coherently without run-time errors:

- **Ableton Live 12** — for all music composition, SFX editing and stem export.
- **FMOD Studio 2.02.11** (or above) — as the audio middleware between Ableton Live and your game engine.
- **Unity 2022 LTS** (or above) OR **Unreal Engine 5** (or above) — pick one, not both.

You must submit two Ableton Live projects: one containing the adaptive backing music, and one containing the in-game SFX. You will then export stems from both projects into a single FMOD Studio project, which will feed your chosen game engine. A short two-minute walkthrough video of the finished game is also required (see Section F).

You must complete the project plan provided in Section A and upload it as part of your submission. Further detail on each deliverable is given in the sections that follow.

Section A – Project plan description

(KU4.1)

Use the sound project plan template below to detail the nature of the samples, sections and states you intend to use for the game. You must submit this template along with your assignment.

The plan must describe the genre and premise of the game (adventure, strategy, action, board game, first-person shooter, racing, puzzle, etc.) and must explain how the music and sound design will respond to gameplay. You must plan for **at least two contrasting musical states** (each with its own genre / tempo feel) and must list **at least six game-action SFX**. You must also indicate which engine you intend to use (Unity *or* Unreal) and justify that choice briefly.

Provide planning details in your own words — marks will be lost for descriptions that are vague, generic or copied from external sources without rewording. The final section of this plan must be a **references** section in a consistent citation style, listing any ready-made samples, libraries or tools you intend to use, with links.

Project Plan Template

Name and Surname: _____

Game Title / Working Title: _____

Target Engine (Unity or Unreal): _____

Target Platform (macOS / Windows): _____

Brief description of the game (genre, setting, mechanics, how the player progresses):

Adaptive Music States — list at least two contrasting states. For each, indicate the musical genre/feel, its intended tempo range, and when it is triggered.

State	Genre / Feel	Tempo range	Trigger / gameplay context
State 1			
State 2			
State 3			
State 4			

Game Action SFX — list at least six in-game sound effects, describe each, and state which gameplay event triggers it.

#	SFX name	Description	In-game trigger
1			
2			
3			
4			
5			

6			
---	--	--	--

References — list any third-party samples, libraries, plug-ins or tutorials used, in a consistent citation style:

KU4.1 — Present a sound brief to explain an audio project		/ 5
Inadequate quality (0 marks): inconsistent information; plan does not reflect work done.		0
Lowest quality (2 marks): not all samples / states described, or descriptions inappropriate.		2
Best quality (5 marks): all states and SFX appropriately described; engine choice justified; references complete.		5



Section B – Backing music template (KU1.1, KU1.2, AA3.1, SE3.2, KU2.1, AA1.3)

Create a project in Ableton Live named **name_surname_class_backing_music.als**. This project will contain all musical material for the adaptive score described in your project plan. You may answer the questions in any order, but each channel, clip and scene must be labelled clearly so your work can be identified during marking.

Questions

1. Structure your project around the adaptive states listed in Section A. Create the project using an approach that clearly separates your musical states — you may use Session View scenes, Arrangement View sections, or a combination of both, but each state must be named after the state it represents (for example *Exploration*, *Build*, *Combat*, *Resolution*). Apply color-coding to distinguish states visually. (KU1.1)
2. Differentiate between an **audio channel**, a **MIDI channel** and a **return track**. Write one paragraph on each, making clear what its function is within an Ableton Live project. Illustrate each explanation with a worked example drawn from your own project. (KU1.1)
3. Create a **customised Drum Rack** (you may not use a stock Ableton Drum Rack preset). The Drum Rack must be built from at least four original percussion samples, and must be used to produce the following MIDI clips: (KU1.2)
 - a. Two different drum clips, each belonging to a different musical state from your sound brief.
 - b. A drum break / fill clip at least two beats long, used to link two contrasting states.

- c. Drum clips may be one measure long, however at least one clip must be at least four measures long.
4. Create a **melodic line** on a channel of your choice (MIDI or audio). The channel must contain the relevant clip(s) for the states in which this melody is heard, and the performance must be properly quantised. **(SE3.2)**
5. Create a **MIDI bass channel** using an instrument of your choice and write a bass line that follows the melody in Question 4. The bass line must be properly quantised and must be audibly distinct across the different adaptive states. **(SE3.2)**
6. Create a **MIDI channel that hosts a third-party VST instrument** of your choice (free VSTs available via Moodle / VLE are acceptable). **(AA3.1)**
 - a. Apply at least two MIDI clips on this channel, each used in a different state as per your sound brief.
 - b. Apply appropriate MIDI effects that help this channel support the adaptive backing track. Document each effect briefly.
7. Create a **chord / filler channel**. This channel must contain an instrument such as keys, guitar or synth, and must hold either a MIDI or an audio clip that complements the backing music. **(KU2.1)**
 - a. The channel must be at least as long as the melody channel.
 - b. Chord variations must be introduced so the listener stays engaged across the duration of the clip.
 - c. The clip must be properly quantised.
8. Create a further MIDI clip on another VST channel. Design a **new synth patch from an initialised state** (using a synthesiser of your choice that supports user patches) and use this patch to reinforce the mood of one of your states. **(SE3.2)**
 - a. Start from an initialised patch and use the oscillator / mixer / filter sections to shape the core tone.



- b. Use the modulation matrix / LFO / envelope sections to add movement, then save the patch and recall it in your Live set.
 - c. Apply a Max-for-Live MIDI effect over part or all of the clip, and explain why it supports the musical state.
9. Apply the following EQ filters on tracks inside your Drum Rack chain (you may route drum pads to individual return chains within the rack): **(KU3.3)**
 - a. A high-pass filter.
 - b. A low-pass filter.
 - c. A band-pass filter.

Do not simply apply the filters — justify each choice by describing which frequencies it targets and why the decision supports the overall mix.

KU1.1 — Explain track composition in a Digital Audio Workstation		/ 5
Q1 — Inadequate (0): inconsistent information.		0
Q1 — Best (2): scenes/sections created and configured correctly.		2
Q2 — Inadequate (0): inconsistent information.		0
Q2 — Best (3): each channel type clearly explained with examples.		3

KU1.2 — Identify and order different sounds within the frequency spectrum		/ 5
Q3a (2 marks): correctly answered as per question.		2
Q3b (1 mark): correctly answered as per question.		1
Q3c (2 marks): correctly answered as per question.		2

AA3.1 — Produce realistic sounds with the use of VSTs / plug-ins		/ 7
Q6 — Low (1): a VST is properly hooked up and used.		1
Q6a (3 marks): correctly answered as per question.		3

Q6b (3 marks): correctly answered as per question.

3

SE3.2 — Manage sound improvements and implementations within a context

/ 10

Q4 — Best (2): correctly answered as per question.

2

Q5 — Best (2): correctly answered as per question.

2

Q8a (2 marks): correctly answered as per question.

2

Q8b (2 marks): correctly answered as per question.

2

Q8c (2 marks): correctly answered as per question.

2

KU2.1 — Identify and report different sounds in a track

/ 5

Q7a (1 mark): correctly answered as per question.

1

Q7b (3 marks): correctly answered as per question.

3

Q7c (1 mark): correctly answered as per question.

1

AA1.3 — Apply equalisation filters using pillars and pre-sets

/ 7

Q9a (2 marks): correctly answered as per question.

2

Q9b (2 marks): correctly answered as per question.

2

Q9c (3 marks): correctly answered as per question.

3



Section C – Mixing, Routing & Effects for the Backing Music (KU1.4, AA2.2, AA3.4, KU3.3)

This section is concerned with how your adaptive backing music is **mixed, routed and processed**. All work is carried out inside the backing-music Ableton project created in Section B.

10. Apply **subtractive equalisation** across the channels in this project. Use an appropriate analyser device to examine the overall EQ on the main mix, and make sure that instruments are not fighting for the same frequency range. Provide screenshots of the resulting frequency spectrum and explain your reasoning for each cut or boost. (KU1.4)
11. Produce an **arrangement** of your adaptive backing music, using labels / locators that match the states you defined in Section A: (AA2.2, AA3.4)
 - a. Use a compressor with **sidechain** on a channel of your choice, and justify why the ducking is appropriate for that state. (AA3.4)
 - b. Use at least three other audio effects, in context. Document each effect and explain why it is necessary. (AA3.4)
 - c. Apply at least four effect automations in Arrangement View. Each automation must serve the gameplay moment it accompanies. (AA2.2)
 - d. Use at least two effects routed through **return tracks**, and apply those returns via sends on your channels. (AA2.2)
 - e. Ensure that your overall mix never peaks above **-0.5 dBs** on the master track when the master fader is at 0 dB position. Use appropriate processing to avoid clipping and justify your chain. (AA2.2)

12. Choose **two** of the following Ableton Live devices, apply each within your project, and describe the parameter modifications you made and their musical purpose:

(KU3.3)

- a. Arpeggiator MIDI device.
- b. Chord MIDI device.
- c. Roar audio effect.
- d. Auto Filter audio effect.

KU1.4 — Examine parametric equalisation within a context	/ 5
Q10 — Lowest (1): EQ used per channel.	1
Q10 — Good (3): proper use of EQ filters per channel.	3
Q10 — Best (5): good quality AND correct documentation provided.	5

AA3.4 — Investigate the use of compressors / expanders within an audio track	/ 7
Q11 — Lowest (1): labels correctly provided as per sound brief.	1
Q11a (2 marks): correctly answered as per question.	2
Q11b (4 marks): correctly answered with documentation provided.	4

AA2.2 — Apply routing processes between channels	/ 7
Q11c (2 marks): correctly answered as per question.	2
Q11d (1 mark): correctly answered with documentation provided.	1
Q11e (4 marks): correctly answered with justification and documentation.	4

KU3.3 — Distinguish / clarify the need of audio and/or MIDI effects	/ 5
Q12 — Lowest (1): one device used.	1
Q12 — Good (2): two devices used.	2
Q12 — Best (5): two devices used AND proper descriptions provided.	5

Section D – In-game SFX and Fmod (KU4.2, SE4.5)

This part of the assignment focuses on the **in-game sound effects** and is delivered as a *separate* Ableton Live project. Create a new project named **name_surname_class_sfx.als**. This project must contain only the samples listed in your sound brief (Section A) — additional samples will not be accepted.

Place each SFX in an appropriate audio channel and label it clearly. You must also include, inside your submission, an **original_sfx** directory containing untouched copies of every source SFX used, so that the unprocessed material can be verified during marking.

Your SFX will later be exported as stems and consumed by your FMOD project (Section E). **Samples cannot be imported into FMOD unless they were exported from Ableton Live in this project.**

Questions

1. Apply **warping techniques** to at least one SFX sample. Use appropriate time-stretching algorithms in Ableton Live so that the resulting sound matches the intended game setting. **(SE4.5)**
2. Apply audio effects to all SFX samples: **(SE4.5)**
 - a. Use Ableton Live audio effect devices to enhance the sonic soundscape of each SFX and embrace the mood of the intended game setting.
 - b. Use audio effect devices to keep the perceived loudness of the SFX samples reasonably uniform, avoiding jarring volume contrasts between cues while keeping the game context in mind.

- c. Briefly describe the intended game setting and explain your decisions for questions (2a) and (2b).
3. Provide a **write-up** describing the use of **vertical re-mixing** and **horizontal re-sequencing**. Explain each technique and its role in dynamic music production. Then establish which method (or combination) you will use for your game, and justify the decision against the gameplay you described in Section A. (KU4.2)

SE4.5 — Justify different techniques learnt and present alternatives or improvements	/ 10
Q1 (2 marks): correctly answered as per requirements.	2
Q2a (3 marks): 0.5 marks per correct SFX.	3
Q2b (3 marks): 0.5 marks per correct SFX.	3
Q2c (2 marks): 1 mark per correct explanation.	2

KU4.2 — Identify the role of interactive composition in dynamic music	/ 5
Q3 — Lowest (1): HRS or VRM correctly explained.	1
Q3 — Better (2): HRS and VRM both correctly explained.	2
Q3 — Best (5): correct explanations AND chosen method properly justified for the game.	5

Section E – Fmod & Game Implementation (AA4.3, SE4.4)

In this section you will build a **single FMOD Studio project** that hosts **both** the adaptive music and the in-game SFX, and you will integrate it into a playable prototype inside **Unity or Unreal Engine** — **pick one engine only**, as declared in your project plan.

Your FMOD project must consume stems exported from the two Ableton Live projects. No assets may be added to FMOD that did not originate in Ableton Live. Music events must live under a **Music** folder/bank, and SFX events under an **SFX** folder/bank, clearly labelled.

You are not required to ship the game itself — a small vertical slice that demonstrates your audio system is sufficient. However, the prototype must be playable by the lecturer long enough for the two-minute walkthrough video (Section F).

Questions

1. Create the appropriate **music events** in FMOD under the Music folder. Apply re-mixing and/or re-sequencing methods as appropriate for your adaptive states. In your write-up, demonstrate how you applied each of the following techniques: **(AA4.3)**
 - a. Transition and loop regions.
 - b. Parameters and conditions (including the parameters exposed to your game engine).
 - c. Transition timelines.
 - d. Probability / chance.
2. Use appropriate FMOD techniques in your game-music folder to achieve: **(SE4.4)**
 - a. Smooth transitions between all the labelled regions of your adaptive score.
 - b. At least one instance of **vertical remixing** driven by a parameter.

3. **Complete SFX package in the game.** Using FMOD events you created for every SFX listed in your sound brief, implement the full in-game sound package inside your chosen engine. **(SE4.4)**
- In **Unity**, wire the FMOD Studio Listener, Event Emitters and trigger zones (or scripted one-shots) to the gameplay events described in your project plan. In **Unreal**, wire the FMOD Audio Component, FMOD Event Emitters and Blueprint / C++ triggers to the equivalent events.
 - Every SFX from your brief must fire in response to an actual gameplay event (collision, input, state change, timer, etc.) — not a scripted timeline playback.
 - At least one SFX must use a **3D spatialised emitter** and at least one SFX must use a **parameter-driven variation** (e.g. a parameter controlling pitch, volume, or event instance selection).
 - In your write-up, justify how Ableton Live and FMOD together make your score adaptive, and explain how your engine integration makes the SFX package respond to gameplay.

AA4.3 — Develop automation in an audio project		/ 7
Q1a (2 marks): transition and loop regions correctly implemented.		2
Q1b (2 marks): parameters and conditions correctly implemented.		2
Q1c (2 marks): transition timelines correctly implemented.		2
Q1d (1 mark): probability / chance correctly implemented.		1

SE4.4 — Part 1 — Evaluate the use of proper sound dynamics within a context (FMOD music)		/ 4
Q2a (2 marks): smooth transitions between labelled regions.		2
Q2b (2 marks): at least one working instance of vertical remixing.		2



Section F – Two minute walk-through video (AA4.3, SE4.4)

A **two-minute maximum** walkthrough video is required as the final deliverable. This video is your opportunity to demonstrate the finished audio system in action and to explain the design decisions that sit behind it.

Requirements

- **Format:** MP4 or MOV, 1080p minimum, stereo audio at 48 kHz. Maximum duration: **120 seconds**. Anything longer than 2:00 will be cut off at the two-minute mark for marking.
- **Scope:** a live screen capture of *your* game running in your chosen engine (Unity or Unreal). Pre-rendered cinematics are not acceptable.
- **Content (all of the following must be visible/audible within the two minutes):**
 - A brief introduction naming you, your game, your chosen engine and the adaptive states you implemented.
 - At least two contrasting musical states playing live in the game, including at least one transition between them, clearly audible.
 - At least one demonstration of **vertical remixing** being driven by a gameplay parameter.
 - The complete SFX package responding to gameplay events — show at least the majority of the SFX from your sound brief firing in response to real in-game actions.
 - A short voice-over (or captioned narration) walking through what the viewer is hearing and why those design choices were made.
- **Submission:** upload the video to a platform accessible to your lecturer (for example OneDrive, Google Drive, YouTube unlisted, or Vimeo) and paste the link in your documentation.

- **Link validity:** make sure the link is working, is not set to *private*, and does not expire.
If the video cannot be opened by the lecturer on the deadline, all marks for this section will be lost.

Paste your video link here: _____

Remember to compress your final submission (both Ableton projects, the FMOD project, your write-ups, the **original_sfx** directory, and your documentation containing the video link) into a single **.zip** file named with your name, surname and class before uploading to Moodle. **Please don't submit your game code binaries and/or source code!**

SE4.4 — Part 2 — Evaluate the use of proper sound dynamics within a context (in-game demonstration) / 6

Inadequate (0): inconsistent information or video missing / unplayable.	0
Good (2): a reasonable number of game sounds can be heard alongside the music.	2
Good (3): a reasonable number of game sounds can be heard alongside the music AND justification is provided.	3
Best (6): all game sounds and music are heard in-game; walkthrough narration clearly explains design choices; video stays within two minutes.	6

End of Assignment Brief